STAYEASY

Final Presentation & Demo Runbook

Flutter + Supabase + Gemini + 360 Room Preview

Student	Aniket Rai
Program	B.Tech CSE 2024-28
Semester	5
Purpose	Final project demonstration and viva rehearsal

Presentation goal

Demonstrate StayEasy as a secure, explainable and modern hotel-booking system - not just a UI prototype.

Quick Pre-Presentation Checklist

Run this 10 minutes before your presentation and keep the browser tabs ready.

```
cd /Users/aniketrai/Desktop/SEM5/StayEasy

flutter run -d web-server \
  --web-port 8081 \
  --dart-define-from-file=config/dev.json
```

```
Customer app: http://localhost:8081
Admin:        http://localhost:8081/admin
```

- Demo account signs in successfully.
- Demo account already has admin role.
- Gemini AI Search returns parsed filters.
- Harbour Queen 360 preview loads and interactions work.
- At least one room is available for your chosen dates.
- My Trips loads the authenticated user booking.
- Admin Dashboard loads before the presentation begins.

Live-demo rule

Do not run migrations, change roles, delete bookings, or edit Supabase during the presentation. Use the already verified environment.

Recommended demo timing (8-10 minutes)

Section	Target time
Intro	0:30
Explore + hotel + room	1:30
Review ranking	1:00
Gemini AI Search	1:15
360 preview	1:00
Booking + My Trips	1:45
Admin	1:15
Architecture + closing	0:45

1. Opening - 30 seconds

Start on the Explore/Home screen.

> *Good morning. My project is StayEasy, a hotel booking application developed using Flutter with Supabase as the backend.*
>
> *StayEasy supports hotel discovery, room browsing, date-based availability, secure booking, authenticated user trips, season-aware review ranking, Gemini-powered natural-language search, grounded AI review insights, an interactive 360-degree room preview, and a role-based admin portal.*
>
> *The frontend is built in Flutter using Material 3, Riverpod for state management, and go_router for navigation. Supabase handles authentication, PostgreSQL, Row Level Security, RPCs, and Edge Functions.*

Pacing
Keep the introduction under 30-40 seconds. The strongest part of the presentation is the live product and architecture explanation.

2. Explore / Home

Point to the destination, dates, guest count, hotel cards and AI search entry point.

> *The user can browse without signing in. The application currently uses Indian destinations and INR pricing.*
>
> *The interface is responsive, so the same Flutter code adapts between mobile, tablet and desktop layouts.*

3. Hotel Details

Open a hotel and briefly show hotel information, amenities, ratings, reviews and room options.

> *Selecting a property opens the hotel details screen, where the user can inspect hotel information, amenities, ratings, reviews, and available room types.*

Then open the room you use for the 360 demo, preferably Harbour Queen.

> *Room information is fetched from Supabase rather than being hardcoded into the UI.*

4. Room Details

Point to room name, price, amenities, capacity, reviews, date/booking action and the 360 preview.

> *The room screen contains the booking-related information along with amenities, pricing, reviews, availability actions, and the virtual room preview.*

5. Date-Aware Review Ranking - Signature Feature

Choose October dates if possible and open the Reviews section.

> *One of the key features of StayEasy is date-aware review relevance.*
>
> *A normal hotel application usually sorts reviews only by newest or highest rated. StayEasy also considers the traveller's intended stay period.*
>
> *If the user plans to stay in October, reviews from October or the same seasonal period receive greater relevance when the remaining factors are comparable.*

Key line
The review ranking is deterministic Dart logic. Gemini does not decide the review order.

> *That keeps the ranking predictable, testable and explainable. AI is only used where language understanding provides value.*

6. Gemini AI Search - Show This Live

Return to Home and use this exact demo-safe query:

```
Find me a hotel in Mumbai for two people in October.
```

Click Interpret request. When the parsed filters appear, point to Mumbai, 2 guests and October.

> *Instead of manually configuring every filter, the user can describe the trip in natural language.*
>
> *The Flutter application sends the request to a Supabase Edge Function. The Edge Function securely calls Google Gemini and asks it to convert the request into validated structured filters.*
>
> *Gemini does not directly choose the hotel and it does not invent properties. It only interprets the user's intent.*
>
> *The actual hotel matching is still performed using StayEasy's repository and real Supabase data.*

If asked: Where is the Gemini API key?

> *The Gemini API key is never stored in the Flutter application. It exists only as a Supabase Edge Function secret.*

If AI temporarily fails

> *The normal search remains fully usable even if the AI provider is temporarily unavailable. AI is an enhancement rather than a dependency for the core booking experience.*

Important
Use the broad Mumbai / two people / October prompt during the live demo. The earlier price + balcony + quiet + Wi-Fi prompt is too restrictive for the seed inventory and may correctly return zero matching hotels.

7. AI Review Insights - Optional Live Demo

Gemini is also used for review insights, but the model only receives selected review evidence.

The backend validates the review IDs returned by the model, so the generated insight must remain grounded in reviews that actually exist.

If there is not enough relevant evidence, StayEasy returns an insufficient-evidence state instead of forcing the AI to generate a summary.

If the UI displays insufficient evidence, explain it as an intentional safety behavior:

Here the system deliberately avoids generating a conclusion because there is not enough evidence.

8. 360-Degree Virtual Room Tour

Open Harbour Queen -> 360 Preview. Demonstrate in this order:

1. Drag the panorama.
2. Zoom.
3. Open a hotspot.
4. Switch Bedroom -> Bathroom -> Balcony.
5. Open fullscreen.
6. Close fullscreen and return to Room Details.

StayEasy also provides an interactive 360-degree room preview.

This is a spherical panorama rather than a horizontally scrolling photograph.

Hotspots are positioned using spherical coordinates, so they stay associated with the correct area while the panorama is moved.

The user can inspect multiple spaces of the room before making a booking decision.

The three demonstration panoramas are stored locally to keep the academic demo reliable and independent of an external panorama server.

9. Authentication

Proceed toward booking/payment. Browsing can remain guest-friendly, but booking requires authentication.

Browsing and AI Search are available to guests, but authentication is required before creating a booking.

Bookings are tied to the authenticated Supabase user so that each customer can securely access only their own trips.

Do not demo email confirmation live
Keep the verified account signed in or ready. Only demonstrate the email verification flow if an examiner specifically asks.

10. Booking Flow

Choose known-working future dates and proceed through Booking Details -> Booking Summary -> Confirm -> Success.

On the summary screen, point out selected dates, guests, nightly price, taxes, discount if shown, and final total.

The displayed booking interface is handled by Flutter, but the client is not trusted to create the authoritative booking itself.

StayEasy uses a Supabase RPC for booking. The server validates the authenticated user, room capacity, active room status, date availability and inventory.

The server calculates the authoritative pricing and reduces inventory atomically.

This prevents the Flutter client from simply modifying values such as the booking price before sending the request.

If asked about concurrent bookings

The booking RPC locks and validates the relevant inventory before decrementing it, reducing the risk of two users booking the last unit at the same time.

11. My Trips

Open My Trips and show the newly created booking.

My Trips displays bookings belonging to the current authenticated user.

Access is enforced both through PostgreSQL permissions and Supabase Row Level Security.

If you choose to show cancellation:

Cancellation is also handled through a server-side RPC. An eligible cancellation restores the corresponding inventory.

12. Admin Portal

```
http://localhost:8081/admin
```

Use the already promoted demo admin account. Show pages in this order: Dashboard, Hotels, Rooms, Availability, Bookings, Reviews.

Dashboard

StayEasy also includes an admin portal in the same Flutter codebase.

The route is protected using role-based authorization. A normal user is redirected away from the administrative interface.

Hotels

Administrators can manage the hotel catalogue.

Rooms

Rooms are linked to their hotels and contain their capacity, pricing and accommodation information.

Availability

Availability is stored per room and date, allowing the booking backend to reason about inventory over a requested stay period.

Bookings

Administrators can inspect the wider booking dataset, while a customer is restricted to their own bookings.

Reviews

Review data is stored separately from the ranking algorithm. This allows the deterministic ranking layer and Gemini insight layer to consume the same verified source data.

Admin demo safety
Avoid editing or deleting production/demo records in the admin portal unless an examiner explicitly asks. Showing real data and explaining the controls is safer.

13. Architecture Explanation

The application follows a feature-oriented Flutter architecture. Riverpod manages application state, go_router manages navigation, and reusable design tokens keep the interface consistent.

Supabase provides authentication, PostgreSQL, Row Level Security and secure RPCs.

Gemini is isolated behind Supabase Edge Functions, so no AI credential is shipped with the client.

AI interprets natural language and summarizes evidence, while critical operations such as pricing, inventory, authorization and review ranking remain deterministic.

Architecture flow

Flutter UI
Riverpod controller
Repository
Supabase boundary
PostgreSQL + RLS
RPC booking logic
Edge Functions -> Gemini

Core design philosophy

- Critical logic -> deterministic and/or server-side
- AI language tasks -> Gemini
- Security -> database grants + RLS + authenticated RPC
- Secrets -> Edge Functions only
- UI -> Flutter

14. Closing - 20 seconds

The goal of StayEasy was not simply to build another hotel-booking UI. The project combines secure booking infrastructure with explainable personalization, controlled use of generative AI, and an immersive room preview.

The result is a Flutter application where AI improves usability without being trusted with critical business logic.

Thank you.

Emergency Demo Recovery

AI says unavailable

> *AI is isolated from the normal search pipeline, so provider failure does not prevent the user from searching normally.*

Continue with regular search.

AI gives no matching hotel

> *Gemini successfully interpreted the request, but the repository found no real room satisfying every constraint. StayEasy deliberately does not fabricate results.*

Then use the safe prompt:

```
Find me a hotel in Mumbai for two people in October.
```

My Trips does not update immediately

Refresh the page once. Do not create multiple duplicate bookings.

> *The booking has already been confirmed by the backend; this page retrieves the user's booking history separately.*

360 preview takes time to open

> *The panorama asset is higher-resolution than the standard hotel images because it must cover the entire viewing sphere.*

Admin redirects to Home

The session is not seeing an admin role. Do not debug this live. Keep a verified admin session ready before presenting.

High-Probability Viva Questions & Answers

Why Flutter?

Flutter allowed me to maintain one responsive UI codebase while targeting mobile and web. It also made it practical to build both the customer experience and admin portal with shared components and models.

Why Riverpod?

Riverpod keeps state outside the widgets, supports dependency injection and asynchronous states cleanly, and makes repository/controller logic easier to test.

Why Supabase?

I needed authentication, PostgreSQL, Row Level Security, RPCs and serverless functions. Supabase provides those within one backend while still giving me direct relational database capabilities.

Why PostgreSQL instead of Firebase?

The project has relational entities such as hotels, rooms, bookings, availability and reviews. PostgreSQL gives explicit relationships, transactional operations and server-side functions, which fit booking logic well.

What is RLS?

Row Level Security is database-level authorization. For example, a customer can read their own bookings rather than every booking simply because they are authenticated.

Is RLS enough?

No. PostgreSQL table privileges and RLS work together. A role first needs permission to perform an operation, then RLS determines which rows that operation can affect.

Why use RPC for booking?

Because booking requires multiple related operations to succeed together: validate availability, validate the room, calculate authoritative totals, create the booking and decrement inventory. A server-side database function can perform that atomically.

Why not calculate price in Flutter?

Flutter can display an estimate, but the client is not trusted. The backend computes the authoritative total so changing the client cannot change the real booking price.

Where is your Gemini API key?

Only in Supabase Edge Function secrets. It is never embedded in Flutter.

What exactly does AI Search do?

It converts natural language into a restricted structured search schema. The application repository performs the actual hotel search.

Can Gemini invent a hotel?

Not through this architecture. Gemini does not return the hotel catalogue; it returns filters. Hotels come from StayEasy data.

What if Gemini is down?

The application gracefully falls back to regular search. Core functionality does not depend on AI.

Why deterministic review ranking?

Ranking impacts what evidence the user sees first. Keeping it deterministic makes it repeatable, testable and explainable.

Then what does Gemini do with reviews?

It summarizes a selected evidence set. Returned source review IDs are validated by the backend.

What if there are too few reviews?

StayEasy returns insufficient evidence instead of asking the model to fabricate a meaningful conclusion.

How is 360 implemented?

The room uses equirectangular panorama images projected onto a spherical viewer. The user changes the viewing direction by dragging, and hotspots use spherical coordinates.

Are those actual hotel panoramas?

They are appropriately licensed demonstration panoramas used for the prototype. The architecture can replace them with property-specific panorama assets.

How do you prevent double booking?

The server-side booking function validates and locks inventory as part of the transaction before decreasing availability.

How is cancellation handled?

Cancellation is performed server-side and is idempotent for already-cancelled bookings. Eligible cancellation restores inventory.

How do admin and user accounts differ?

The profile contains a role. The Flutter route guard controls the interface, while backend authorization remains the security boundary.

Could someone manually type /admin?

Yes, which is why hiding the button is not security. The application checks the authenticated role and redirects unauthorized users. Backend policies remain necessary even if someone bypassed the UI.

What is your most unique feature?

The combination of intended stay dates with deterministic review relevance. Instead of showing the same review ordering to every traveller, StayEasy can prioritize feedback that is more representative of the period they intend to visit.

What would you add in future?

Real payment-gateway integration, hotel-owner onboarding, property-specific 360 capture, larger review datasets for stronger AI insights, notification workflows, recommendation personalization and production monitoring.

Final Demo Checklist

Feature	Demo status
Explore/Home	**Verified**
Hotel Details	**Verified**
Room Details	**Verified**
Date-aware review ranking	**Verified**
Gemini AI Search	**Verified**
360-degree room tour	**Verified**
Authentication	**Verified**
Booking	**Verified**
My Trips	**Verified**
Admin Portal	**Verified**

Strongest technical story

Flutter UI -> Riverpod -> Repository -> Supabase. Keep critical business logic deterministic and server-side; use Gemini only for language understanding and evidence-grounded summarization.